SOS & Safety — Panic Alerts, Incidents & Escalation

App: `apps/sos` **Priority:** Mission-critical. SOS record writes are always synchronous. Failures are never swallowed.

Table of Contents

---

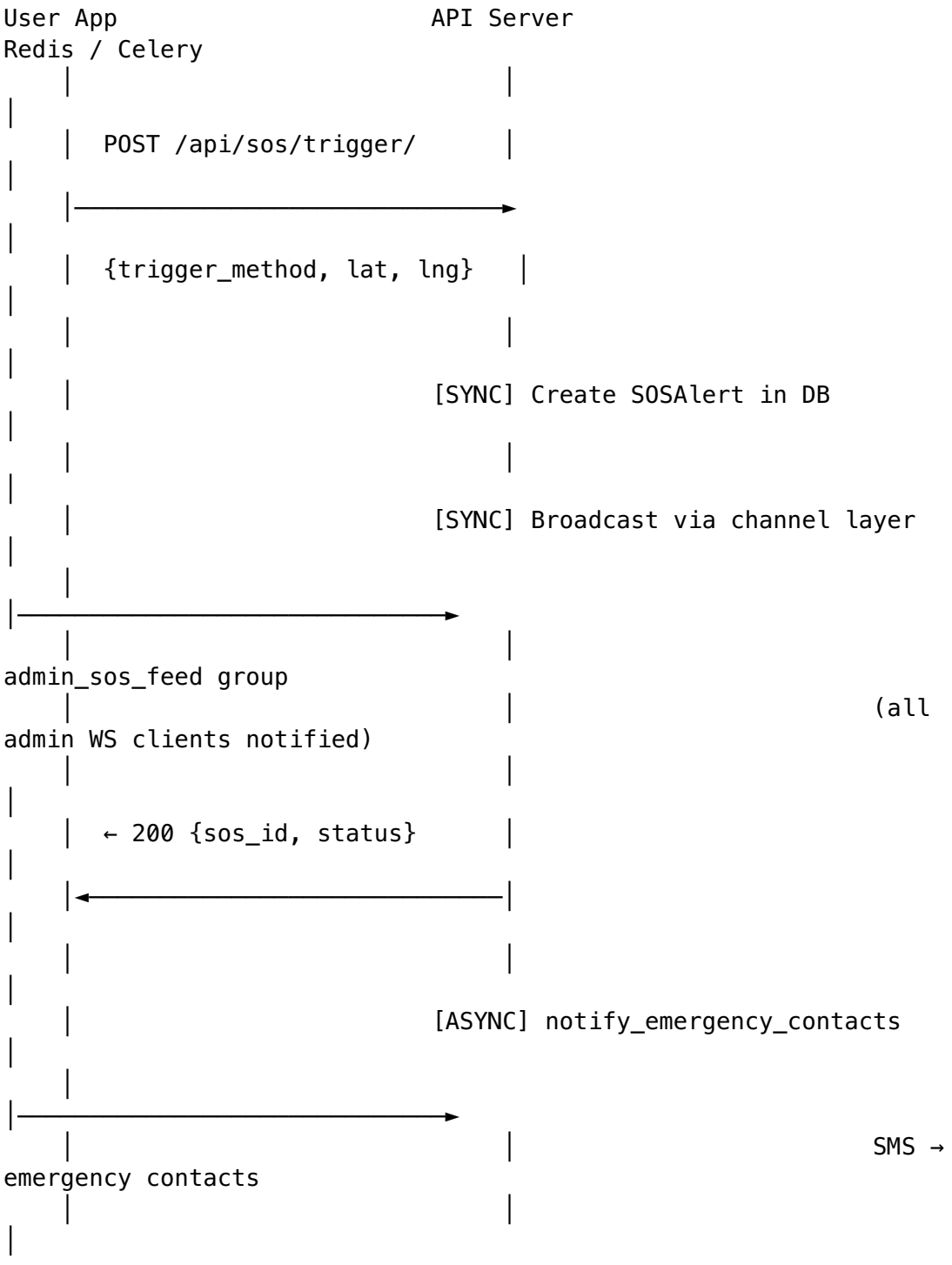
1. SOS System Overview

```
TRIGGERS:
  1. User presses SOS button in app
  2. User shakes phone 3x (accelerometer — handled client-side,
sends same API call)
  3. Guard missed 2 consecutive check-ins (auto-escalation via
Celery)
  4. Guard goes offline during active session (Dead Man's Switch)
  5. Guard presses their own Distress button

ON TRIGGER:
  [Sync — must complete in < 500ms]
  → Write SOSAlert record to DB
  → Broadcast to admin WS group (sos_feed)

  [Async — Celery high_priority queue]
  → SMS to all emergency contacts with GPS location + Google Maps
link
  → Push notification to guard (if booking active)
```

```
  → Push notification to user (SOS acknowledged notification —
once admin acts)
  → If no admin acknowledges in 5 min → escalation task fires

SOS RESOLUTION:
  Admin acknowledges → ACKNOWLEDGED
  Admin dispatches help → RESPONDING
  Admin closes → RESOLVED or FALSE_ALARM
```

---

2. SOS Trigger Flow



```
    │                          [ASYNC] notify_guard_of_user_sos   │
    │
│──────────────────────────────►
    │                               │                          Push
→ guard
    │                               │
│
    │                          [SCHEDULED — 5 min] escalation_check
    │
│──────────────────────────────►
    │                               │                          If
still TRIGGERED → escalate
```

---

3. SOS Service Implementation

```python
# apps/sos/services.py

import logging
from django.utils import timezone
from asgiref.sync import async_to_sync
from channels.layers import get_channel_layer

logger = logging.getLogger(__name__)


class SOSService:

    @staticmethod
    def trigger_sos(user, trigger_method: str, latitude: float,
                    longitude: float, booking=None) -> 'SOSAlert':
        """
        Core SOS trigger method.
        MUST be fast — DB write and WS broadcast are synchronous.
        Everything else is async via Celery.

        This method is called from the REST API view.
        """
        from .models import SOSAlert
        from apps.notifications.tasks import (
            notify_emergency_contacts,
            notify_guard_of_user_sos,
        )
        from .tasks import schedule_sos_escalation

        # STEP 1: Write SOS record (synchronous — must never fail
        silently)
        sos = SOSAlert.objects.create(
            user=user,
            booking=booking,
            trigger_method=trigger_method,
```

```
            latitude=latitude,
            longitude=longitude,
            status='TRIGGERED',
        )

        logger.critical(
            f'SOS TRIGGERED: id={sos.id} user={user.id} '
            f'method={trigger_method} lat={latitude}
        lng={longitude} '
            f'booking={booking.id if booking else None}'
        )

        # STEP 2: Real-time broadcast to admin control room
        (synchronous WS)
        SOSService._broadcast_to_admins(sos)

        # STEP 3: Async — notify emergency contacts
        notify_emergency_contacts.apply_async(
            args=[str(sos.id)],
            queue='high_priority',
            countdown=0,
        )

        # STEP 4: Async — notify guard if session is active
        if booking and booking.status == 'ACTIVE':
            notify_guard_of_user_sos.apply_async(
                args=[str(booking.id)],
                queue='high_priority',
                countdown=0,
            )

        # STEP 5: Schedule escalation if no admin responds within
        5 minutes
        schedule_sos_escalation.apply_async(
            args=[str(sos.id)],
            countdown=300,  # 5 minutes
            queue='high_priority',
        )

        return sos

    @staticmethod
    def _broadcast_to_admins(sos: 'SOSAlert'):
        """Broadcast new SOS to all connected admin WebSocket
        clients."""
        channel_layer = get_channel_layer()
        async_to_sync(channel_layer.group_send)('admin_sos_feed',
        {
            'type': 'sos_alert',
            'payload': {
                'event': 'NEW_SOS',
                'sos_id': str(sos.id),
                'user_id': str(sos.user_id),
```

```
            'user_name': sos.user.display_name,
            'user_phone_masked': sos.user.phone_number[:3] +
    '****' + sos.user.phone_number[-4:],
            'booking_id': str(sos.booking_id) if
    sos.booking_id else None,
            'trigger_method': sos.trigger_method,
            'latitude': float(sos.latitude),
            'longitude': float(sos.longitude),
            'triggered_at': sos.created_at.isoformat(),
            'maps_link': f'https://maps.google.com/?
    q={sos.latitude},{sos.longitude}',
        }
    })

@staticmethod
def acknowledge_sos(sos_id: str, admin_user) -> 'SOSAlert':
    from .models import SOSAlert
    sos = SOSAlert.objects.get(id=sos_id)
    if sos.status == 'TRIGGERED':
        sos.status = 'ACKNOWLEDGED'
        sos.assigned_to = admin_user
        sos.acknowledged_at = timezone.now()
        sos.save(update_fields=['status', 'assigned_to',
    'acknowledged_at'])

        # Notify user that SOS was acknowledged
        from apps.notifications.tasks import
    send_sos_acknowledged_notification

    send_sos_acknowledged_notification.delay(str(sos.user_id))

        # Broadcast status update to admin panel
        SOSService._broadcast_status_update(sos)
    return sos

@staticmethod
def resolve_sos(sos_id: str, admin_user, notes: str,
                is_false_alarm: bool = False) -> 'SOSAlert':
    from .models import SOSAlert
    sos = SOSAlert.objects.get(id=sos_id)
    sos.status = 'FALSE_ALARM' if is_false_alarm else
    'RESOLVED'
    sos.resolved_at = timezone.now()
    sos.resolution_notes = notes
    sos.save()
    SOSService._broadcast_status_update(sos)
    return sos

@staticmethod
def _broadcast_status_update(sos: 'SOSAlert'):
    channel_layer = get_channel_layer()
    async_to_sync(channel_layer.group_send)('admin_sos_feed',
    {
        'type': 'sos_status_update',
```

```
            'payload': {
                'event': 'SOS_STATUS_CHANGE',
                'sos_id': str(sos.id),
                'new_status': sos.status,
                'timestamp': timezone.now().isoformat(),
            }
        })
```

---

4. Check-in System

Guards on long sessions (daily, weekly, monthly) must check in every N hours to confirm they are still active and safe.

```
# apps/sos/tasks.py

from celery import shared_task
from django.utils import timezone
from datetime import timedelta
import logging

logger = logging.getLogger(__name__)

CHECKIN_INTERVAL_HOURS = 2     # Guards must check in every 2
        hours
CHECKIN_WARNING_MINUTES = 15   # Alert user 15 min before checkin
        is due
CHECKIN_ESCALATION_MINUTES = 30  # Escalate if checkin missed by
        30 min


@shared_task(queue='default', name='sos.monitor_session_checkins')
def monitor_session_checkins():
    """
    Runs every 15 minutes (Celery beat).
    Checks all active sessions for missed check-ins.
    """
    from apps.bookings.models import Booking, GuardCheckIn
    from django.db.models import Max

    active_sessions = Booking.objects.filter(
        status='ACTIVE',
        service_type__in=['DAILY', 'WEEKLY', 'MONTHLY'],  # Only
        long sessions need check-ins
    ).select_related('user', 'guard__user')

    now = timezone.now()

    for booking in active_sessions:
        # Find the most recent check-in for this session
        last_checkin =
        GuardCheckIn.objects.filter(booking=booking).aggregate(
```

```
            last=Max('created_at')
        )['last']

        reference_time = last_checkin or
        booking.session_started_at

        if not reference_time:
            continue

        minutes_since_checkin = (now -
        reference_time).total_seconds() / 60
        checkin_due_at_minutes = CHECKIN_INTERVAL_HOURS * 60

        # Check-in overdue by 30+ minutes → escalate
        if minutes_since_checkin >= checkin_due_at_minutes +
        CHECKIN_ESCALATION_MINUTES:
            _escalate_missed_checkin(booking,
        minutes_since_checkin)

        # Check-in overdue (missed) → alert user
        elif minutes_since_checkin >= checkin_due_at_minutes:
            _alert_missed_checkin(booking)


def _alert_missed_checkin(booking):
    """Alert user that their guard has missed a check-in."""
    from apps.notifications.tasks import notify_checkin_missed
    from apps.notifications.models import NotificationLog

    # Only send once per missed window (check if already notified)
    already_notified = NotificationLog.objects.filter(
        recipient=booking.user,
        notification_type='CHECKIN_MISSED',
        created_at__gte=timezone.now() -
        timedelta(hours=CHECKIN_INTERVAL_HOURS),
    ).exists()

    if not already_notified:
        notify_checkin_missed.delay(str(booking.id))


def _escalate_missed_checkin(booking):
    """Create an SOS alert due to missed check-in escalation."""
    from apps.sos.services import SOSService

    logger.warning(
        f'Auto-escalating SOS for booking {booking.id}: '
        f'guard {booking.guard_id} missed check-in significantly'
    )

    SOSService.trigger_sos(
        user=booking.user,
        trigger_method='AUTO_CHECKIN',
        latitude=float(booking.service_latitude),
```

```
            longitude=float(booking.service_longitude),
            booking=booking,
        )
```

---

5. Dead Man's Switch

If a guard's device goes completely offline during an active session, the system escalates automatically.

```
# apps/sos/tasks.py (continued)


@shared_task(queue='default', name='sos.check_guard_offline')
def check_guard_offline_sessions():
    """
    Runs every 5 minutes (Celery beat).
    Detects guards who stopped sending location updates during
        active sessions.
    """
    from apps.bookings.models import Booking
    from apps.guards.models import GuardProfile
    from django.utils import timezone

    OFFLINE_THRESHOLD_MINUTES = 10  # Alert if no update for 10
        minutes

    threshold_time = timezone.now() -
        timedelta(minutes=OFFLINE_THRESHOLD_MINUTES)

    # Active sessions where guard's last_location_update is too
        old
    at_risk_bookings = Booking.objects.filter(
        status='ACTIVE',
        guard__last_location_update__lt=threshold_time,
        guard__is_online=True,  # Guard thinks they're online but
        isn't sending
    ).select_related('user', 'guard__user')

    for booking in at_risk_bookings:
        logger.error(
            f'Dead Man Switch: guard {booking.guard_id} offline
        during '
            f'active session {booking.id}'
        )

        # Mark guard as offline
        GuardProfile.objects.filter(id=booking.guard_id).update(is_online=False)

        # Create SOS alert
        from apps.sos.services import SOSService
        SOSService.trigger_sos(
```

```
            user=booking.user,
            trigger_method='GUARD_OFFLINE',
            latitude=float(booking.service_latitude),
            longitude=float(booking.service_longitude),
            booking=booking,
        )
```

---

6. Guard Distress Flow

```
# apps/sos/views.py (partial)

class GuardDistressView(APIView):
    """
    POST /api/sos/guard-distress/
    Called when a guard presses their own distress button.
    """
    permission_classes = [IsAuthenticated, IsGuard]

    def post(self, request):
        lat = request.data.get('latitude')
        lng = request.data.get('longitude')
        booking_id = request.data.get('booking_id')

        booking = None
        if booking_id:
            from apps.bookings.models import Booking
            try:
                booking = Booking.objects.get(
                    id=booking_id,
                    guard=request.user.guard_profile,
                    status='ACTIVE'
                )
            except Booking.DoesNotExist:
                pass

        # Trigger SOS on behalf of the guard (guard is the
        distressed party)
        sos = SOSService.trigger_sos(
            user=request.user,  # Guard's user account
            trigger_method='GUARD_DISTRESS',
            latitude=float(lat),
            longitude=float(lng),
            booking=booking,
        )

        # Also alert the user in the active session
        if booking:
            from apps.notifications.tasks import
        notify_user_of_guard_distress
            notify_user_of_guard_distress.delay(str(booking.id))
```

```
        return Response({
            'data': {
                'sos_id': str(sos.id),
                'status': 'TRIGGERED',
                'message': 'Distress alert sent. Control room
        notified.',
            }
        })
```

---

7. Incident Reporting

```
# apps/sos/views.py (partial)

from rest_framework import viewsets
from rest_framework.parsers import MultiPartParser, FormParser,
        JSONParser
from .models import Incident, IncidentEvidence
from .serializers import IncidentCreateSerializer,
        IncidentSerializer


class IncidentViewSet(viewsets.GenericViewSet):
    parser_classes = [MultiPartParser, FormParser, JSONParser]
    permission_classes = [IsAuthenticated]

    def create(self, request):
        """
        POST /api/incidents/
        File an incident report with optional photo/video
        evidence.
        """
        serializer = IncidentCreateSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        from apps.bookings.models import Booking
        booking_id = serializer.validated_data['booking_id']

        try:
            booking = Booking.objects.get(id=booking_id)
        except Booking.DoesNotExist:
            return Response({'error': {'code': 'NOT_FOUND'}},
        status=404)

        # Ensure the filer is a participant
        if booking.user != request.user and (
            not hasattr(request.user, 'guard_profile') or
            booking.guard != request.user.guard_profile
        ):
            return Response({'error': {'code':
        'PERMISSION_DENIED'}}, status=403)
```

```
        incident = Incident.objects.create(
            booking=booking,
            filed_by=request.user,

        incident_type=serializer.validated_data['incident_type'],
            severity=serializer.validated_data['severity'],
            description=serializer.validated_data['description'],
        )

        # Handle file uploads
        evidence_files = request.FILES.getlist('evidence_files')
        for f in evidence_files[:5]:  # Max 5 files
            file_type = 'IMAGE' if
        f.content_type.startswith('image') else 'VIDEO'
            IncidentEvidence.objects.create(
                incident=incident,
                file=f,
                file_type=file_type,
            )

        # Notify admin panel
        from .tasks import notify_admin_new_incident
        notify_admin_new_incident.delay(str(incident.id))

        return Response({
            'data': {
                'incident_id': str(incident.id),
                'status': 'OPEN',
                'message': 'Incident reported. Our team will
         review within 24 hours.',
                'evidence_count': len(evidence_files),
            }
        }, status=201)
```

---

8. Admin Escalation Workflow

```
# apps/sos/tasks.py (continued)

@shared_task(queue='high_priority',
        name='sos.schedule_escalation')
def schedule_sos_escalation(sos_id: str):
    """
    Called 5 minutes after SOS is triggered.
    If still not acknowledged → escalate (send to supervisor,
        increase alert level).
    """
    from apps.sos.models import SOSAlert

    sos = SOSAlert.objects.get(id=sos_id)
```

```
    if sos.status == 'TRIGGERED':
        # Still not acknowledged after 5 minutes — critical
        escalation
        logger.critical(
            f'SOS ESCALATION: {sos_id} not acknowledged after 5
        minutes! '
            f'User: {sos.user_id}'
        )

        # Send escalation SMS to supervisor numbers
        from apps.notifications.services.sms import SMSService
        from django.conf import settings

        for supervisor_phone in settings.SOS_SUPERVISOR_PHONES:
            SMSService.send_sms(
                to_number=supervisor_phone,
                body=(
                    f'URGENT: SOS alert {str(sos_id)[:8]} has not
        been acknowledged '
                    f'for 5 minutes. User:
        {sos.user.display_name}. '
                    f'Location: https://maps.google.com/?
        q={sos.latitude},{sos.longitude}'
                )
            )

        # Schedule second escalation at 15 minutes
        schedule_second_escalation.apply_async(
            args=[sos_id],
            countdown=600,  # 10 more minutes
            queue='high_priority',
        )


@shared_task(queue='high_priority', name='sos.second_escalation')
def schedule_second_escalation(sos_id: str):
    """15 minutes after SOS — auto-escalate to nearest police
        station API."""
    from apps.sos.models import SOSAlert
    from django.conf import settings

    sos = SOSAlert.objects.get(id=sos_id)

    if sos.status in ('TRIGGERED', 'ACKNOWLEDGED'):
        logger.critical(f'SOS CRITICAL: {sos_id} unresolved at 15
        min mark!')
        # Future: integrate with police API / ERSS (Emergency
        Response Support System)
        # For now: alert CTO / on-call engineer
        from apps.notifications.services.email import EmailService
        EmailService.send_email(
            to_email=settings.ONCALL_ENGINEER_EMAIL,
            subject=f'CRITICAL: Unresolved SOS Alert {str(sos_id)
        [:8]}',
```

```
            html_content=f'<p>SOS {sos_id} is unresolved after 15
        minutes. Immediate action required.</p>'
        )
```

---

9. SOS Views & URLs

```python
# apps/sos/views.py

from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from utils.permissions import IsAdminUser
from .services import SOSService
from .serializers import SOSTriggerSerializer, SOSAlertSerializer


class SOSTriggerView(APIView):
    """POST /api/sos/trigger/ — MUST respond in < 1 second."""
    permission_classes = [IsAuthenticated]

    def post(self, request):
        serializer = SOSTriggerSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        booking = None
        booking_id = serializer.validated_data.get('booking_id')
        if booking_id:
            from apps.bookings.models import Booking
            try:
                booking = Booking.objects.get(
                    id=booking_id,
                    user=request.user,
                    status__in=['ACTIVE', 'ARRIVED', 'EN_ROUTE']
                )
            except Booking.DoesNotExist:
                pass

        sos = SOSService.trigger_sos(
            user=request.user,

        trigger_method=serializer.validated_data['trigger_method'],
            latitude=serializer.validated_data['latitude'],
            longitude=serializer.validated_data['longitude'],
            booking=booking,
        )

        return Response({
            'data': {
                'sos_id': str(sos.id),
                'status': 'TRIGGERED',
```

```
                'message': 'Emergency contacts notified. Control
        room alerted.',
                'triggered_at': sos.created_at.isoformat(),
            }
        })


class AdminSOSAcknowledgeView(APIView):
    """POST /api/admin/sos/alerts/{id}/acknowledge/"""
    permission_classes = [IsAdminUser]

    def post(self, request, pk):
        sos = SOSService.acknowledge_sos(pk, request.user)
        return Response({'data': SOSAlertSerializer(sos).data})


class AdminSOSResolveView(APIView):
    """POST /api/admin/sos/alerts/{id}/resolve/"""
    permission_classes = [IsAdminUser]

    def post(self, request, pk):
        notes = request.data.get('resolution_notes', '')
        is_false_alarm = request.data.get('is_false_alarm', False)
        sos = SOSService.resolve_sos(pk, request.user, notes,
        is_false_alarm)
        return Response({'data': SOSAlertSerializer(sos).data})

# apps/sos/urls.py

from django.urls import path
from .views import SOSTriggerView, GuardDistressView,
        IncidentViewSet

urlpatterns = [
    path('trigger/', SOSTriggerView.as_view(), name='sos-
        trigger'),
    path('guard-distress/', GuardDistressView.as_view(),
        name='guard-distress'),
    path('', IncidentViewSet.as_view({'post': 'create', 'get':
        'list'}), name='incidents'),
    path('<uuid:pk>/', IncidentViewSet.as_view({'get':
        'retrieve'}), name='incident-detail'),
    path('<uuid:pk>/evidence/', IncidentViewSet.as_view({'post':
        'add_evidence'}), name='incident-evidence'),
]
```